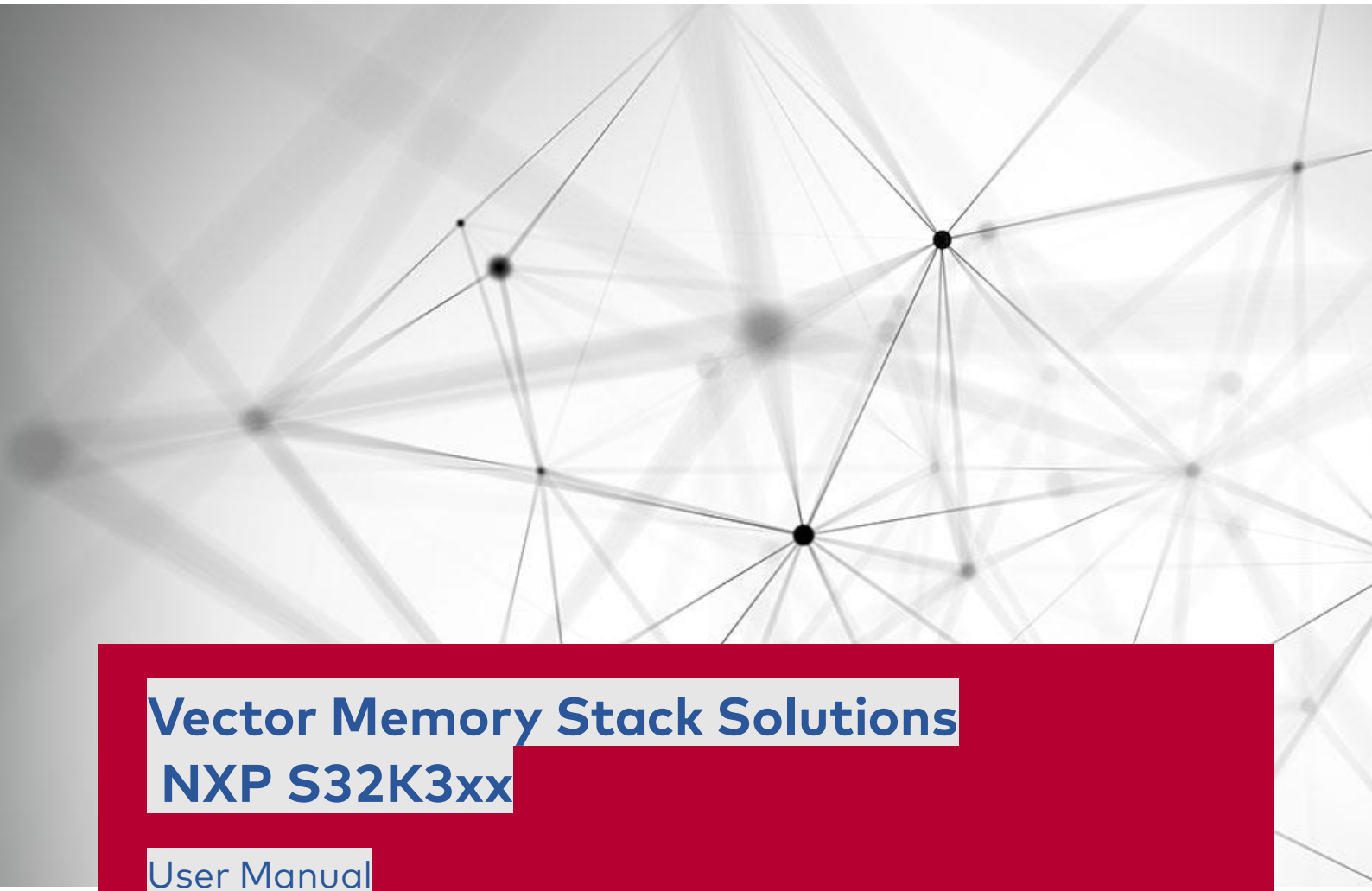
VECTOR



Vector Memory Stack Solutions NXP S32K3xx

User Manual

Tobias Horn

V1.0.1 | Released

Document Information

History

Author	Date	Version	Remarks
Tobias Horn	01.08.2022	1.0.0	Initial version
Tobias Horn	18.10.2022	1.0.1	Updated Chapter 1.11 and Chapter 1.12 and minor changes in Chapter 3



Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

© 2022 Vector Informatik GmbH

Contents

1 Introduction 5
1.1 Application and Audience 5
1.2 Relevant BSW Modules 6
1.3 AUTOSAR Non-volatile Memory Management 6
1.4 AUTOSAR Non- volatile Memory Management - Architecture 7
1.5 AUTOSAR Non- volatile Memory Management - Components 8
1.6 Vector vMem Stack 8
1.7 AUTOSAR vs. MICROSAR Memory Architecture 9
1.8 vMem Stack Components- FlsvMemAccM 9
1.9 vMem Stack Components- vMemAccM 9
1.10 vMem Stack Components – vMem 10
1.11 Overview of Complete Memory Stack with Variants for Fee 11
1.12 Overview of Complete Memory Stack with Variants for Eep 11
1.13 FAQs 12

2 Hardware dependent use cases 13
2.1 Structure 13

3 NXP S32K3xx 14
3.1 General Information 14
3.2 Use Case I: NV Storage Internal Memory 15
3.3 Use Case II: NV Storage External Memory 16
3.4 Use Case III: OTA Internal Memory and NV Storage Internal Memory 17
3.5 Use Case IV: OTA External Memory and NV Storage External Memory 18
3.6 Use Case V: OTA Internal Memory and NV Storage External Memory 19
3.7 Use Case VI: OTA External Memory and NV Storage Internal Memory 20
3.8 Use Case VII: MSR with HSM 21

4 Flash Memory Background Information 22
4.1 Flash Memory Characteristics 22
4.2 Flash Memory Types – NOR 23
4.3 Flash Memory Types – NAND 24
4.4 Memory Types – NAND Flash Implementations 25

5 Glossary and Abbreviations 26
5.1 Abbreviations 26

6 Contact 27

© 2022 Vector Informatik GmbH

Illustrations

Figure 1-1 Relevant BSW Modules 6
Figure 1-2 AUTOSAR Non- volatile Memory Management – Architecture 7
Figure 1-3 AUTOSAR vs. MICROSAR Memory Architecture 9
Figure 1-4 AUTOSAR vs. MICROSAR Memory Architecture 9
Figure 1-5 Overview of Complete Memory Stack with Variants for Fee 11
Figure 1-6 Overview of Complete Memory Stack with Variants for Eep 11
Figure 3-1 Use Case I: NV Storage Internal Memory 15
Figure 3-2 Use Case II: NV Storage External Memory 16
Figure 3-3 Use Case III: OTA Internal Memory and NV Storage Internal Memory 17
Figure 3-4 Use Case IV: OTA External Memory and NV Storage External Memory 18
Figure 3-5 Use Case V: OTA Internal Memory and NV Storage External Memory 19
Figure 3-6 Use Case VI: OTA External Memory and NV Storage Internal Memory 20
Figure 3-7 Use Case VII: MSR with HSM 21
Figure 4-1 Flash Memory Characteristics I/II 23
Figure 4-2 Flash Memory Types – NOR 24
Figure 4-3 Flash Memory Types – NAND 24

Tables

No table of figures entries found.

1 Introduction

1.1 Application and Audience

- This document provides some general background information about the Vector memory solution and some guidance to select the right solution for a specific hardware platform
- Audience
  - Customers
  - Architects and Integrators for the memory stack
- The document is split in four parts
  - General introduction
  - Flash technology background information
  - Solution guidance for specific hardware platforms
  - Indications for SoC platforms
- Since there are strong dependencies between the MICROSAR BSW modules regarding the applicable memory solution, this document considers these dependencies for
  - Non-volatile data storage
  - Security
  - OTA software update

1.2 Relevant BSW Modules

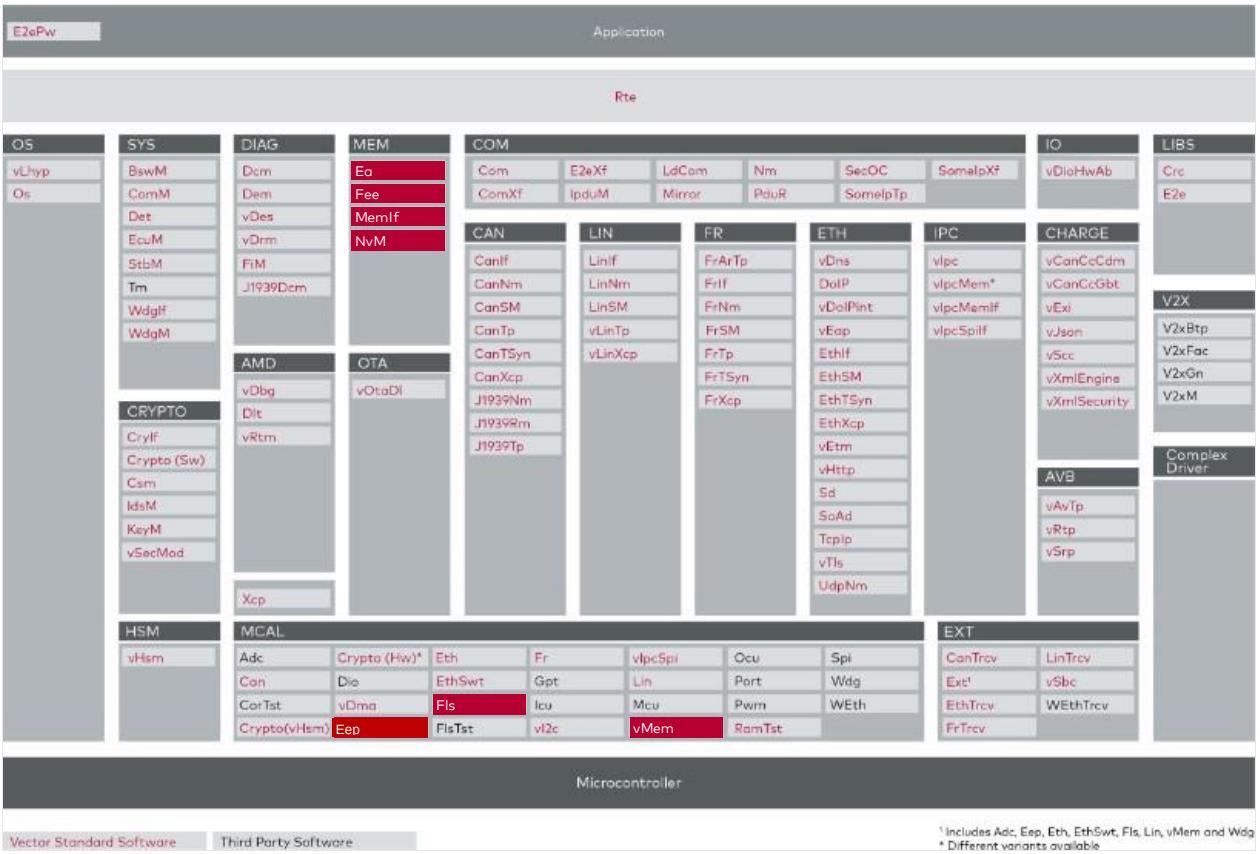


Figure 1-1 Relevant BSW Modules

1.3 AUTOSAR Non-volatile Memory Management

The AUTOSAR memory stack defines basic software modules to permanently store data in non-volatile memory

Typical use cases:

> Storing parameter/calibration data

> Diagnostic fault memory

The AUTOSAR specification addresses non-volatile data storage in data Flash or EEPROM devices

Extension of AUTOSAR NVM stack:

> The Vector memory solution covers the software update use case by means of the vMem stack as the AUTOSAR memory stack specification is not covering software update use case before ASR 22-11

© 2022 Vector Informatik GmbH

1.4 AUTOSAR Non- volatile Memory Management - Architecture

> Depending on the memory device, either an EEEPROM Abstraction (Ea) or a Flash EEPROM Emulation (Fee) is used

> Normally, Fee and Ea are hardware independent except for some special derivatives

> Fls and Eep drivers are typically part of the semiconductor MCAL package, except for external memory drivers which are provided by Vector

> The AUTOSAR memory stack supports usage of Ea and Fee in parallel

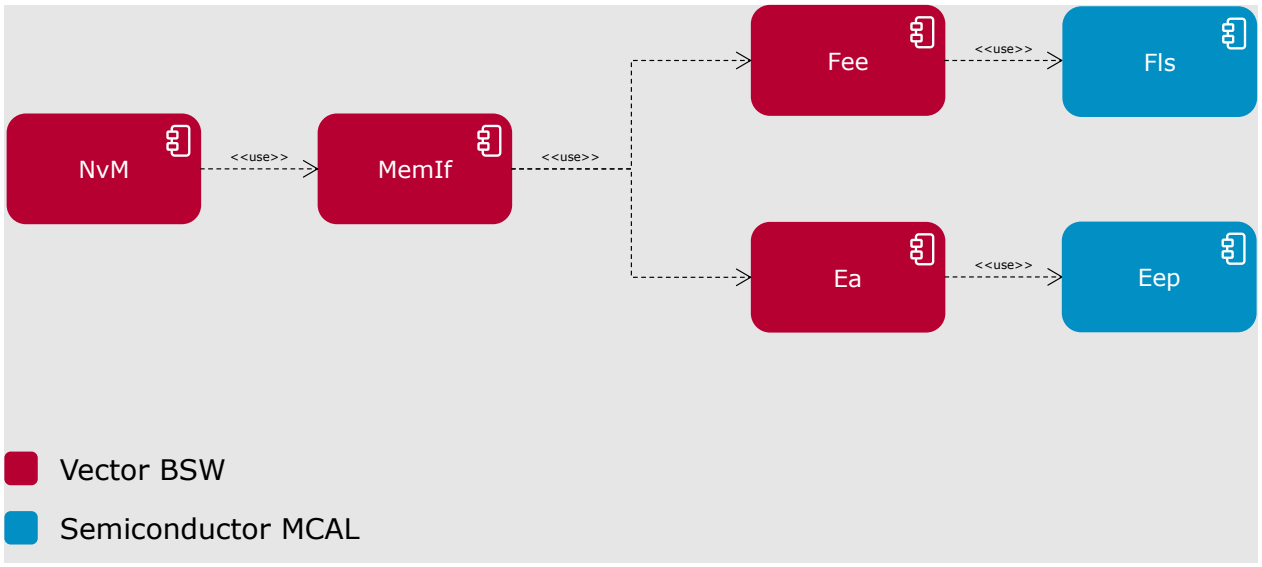


Figure 1-2 AUTOSAR Non- volatile Memory Management – Architecture

© 2022 Vector Informatik GmbH

1.5 AUTOSAR Non- volatile Memory Management - Components

NvM – NVRAM Manager

- Hardware independent storage and maintenance of non-volatile data
- Implements RAM mirror to decouple data access

MemIf – Memory Interface

- Abstraction from the underlying Fee and Ea modules
- Virtual segmentation on a uniform linear address space

Fee – Flash EEPROM Emulation

- Flash hardware abstraction like specific addressing scheme and segmentation

Fls – Flash driver

- Hardware specific implementation of physical Flash access

Ea – EEPROM Abstraction

- Hardware abstraction from the addressing scheme of underlying EEPROM

Eep – EEPROM driver

- Hardware specific implementation of physical EEPROM access

1.6 Vector vMem Stack

The AUTOSAR memory stack specification does not cover code Flash memory access

- OTA software and HSM software update require code Flash write access

New requirements driven by OTA software download use case

- Hardware restrictions of parallel erase/write access to code and data Flash
  - Synchronization of different upper layers like OTA software update manager (vSwUpdM) and Fee necessary
  - Multiprocessor/HSM synchronization
- Multiple memory driver, e.g. internal/external Flash, external EEPROM, RAM



Note
Vector vMem stack addresses code flash access for software update as well as data flash access for non-volatile data storage if no MCAL Fls is available.

1.7 AUTOSAR vs. MICROSAR Memory Architecture

AUTOSAR memory architecture

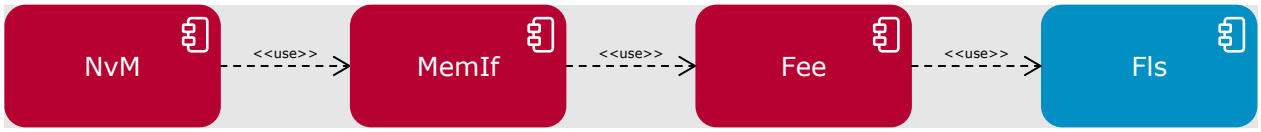


Figure 1-3 AUTOSAR vs. MICROSAR Memory Architecture

MICROSAR memory architecture

> Fls driver is replaced by vMem stack

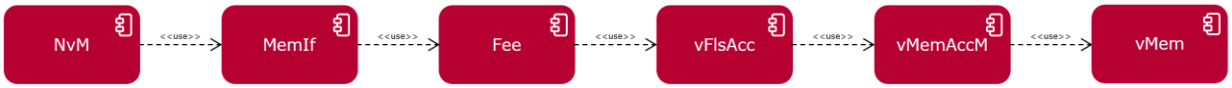


Figure 1-4 AUTOSAR vs. MICROSAR Memory Architecture

1.8 vMem Stack Components- FlsvMemAccM

> Interface component between Fee and vMemAccM provides AUTOSAR Fls interface

> Forward memory access requests to vMemAccM

> Only relevant for NvM/Fee use case

1.9 vMem Stack Components- vMemAccM

> Memory access manager coordinates the memory access of multiple users due to hardware restrictions, e.g.

  > AUTOSAR NvM/Fee

  > OTA software download

  > HSM

> Memory virtualization

> Job management

> Multiple instance memory driver support

© 2022 Vector Informatik GmbH

1.10 vMem Stack Components – vMem

Vector memory driver provides services for accessing internal/external memory devices

- Flash
- EEPROM
- RAM
  - Offers basic functionalities like read, write, erase and job status

vMem driver operates on flash pages and sectors

- Erasing multiple sectors or writing multiple pages is handled by vMemAccM
- vMem must always be used with vMemAccM

© 2022 Vector Informatik GmbH

1.11 Overview of Complete Memory Stack with Variants for Fee

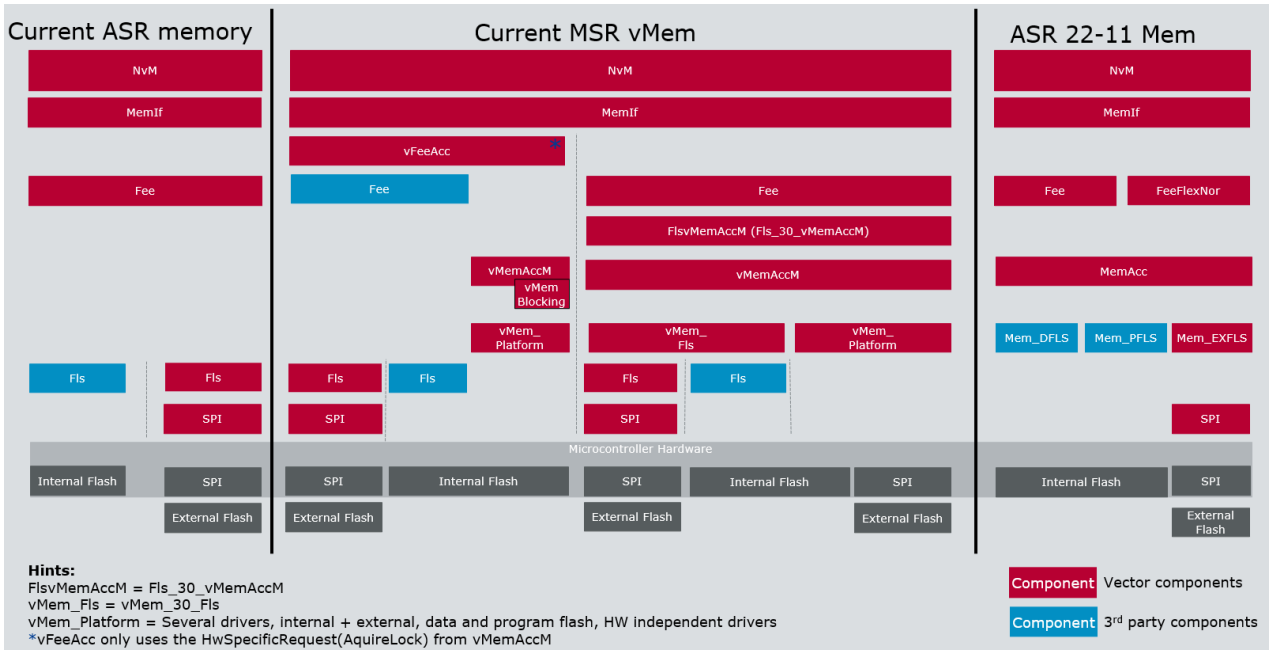


Figure 1-5 Overview of Complete Memory Stack with Variants for Fee

1.12 Overview of Complete Memory Stack with Variants for Eep

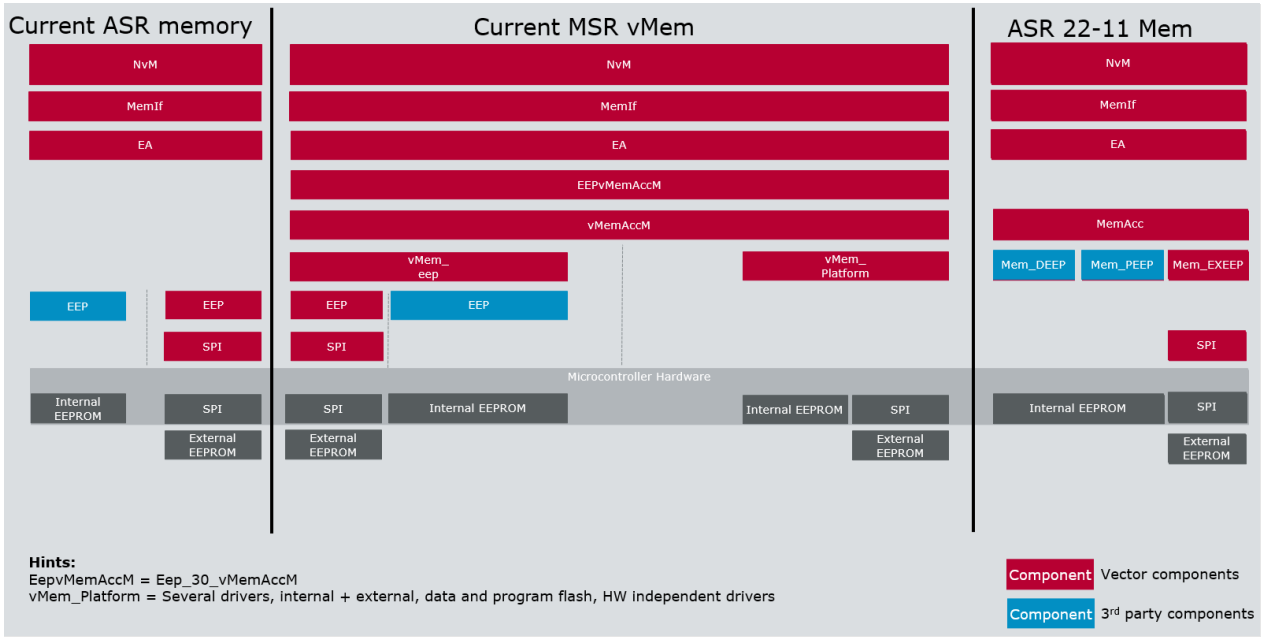


Figure 1-6 Overview of Complete Memory Stack with Variants for Eep

1.13 FAQs

Q: Can MICROSAR MEM be used in CANbedded product?

A: MICROSAR MEM has few dependencies to other modules like DEM, DET as well as CFG5 as generator framework. If these components are stubbed/manually created and CFG5 is part of the delivery, MICROSAR MEM can be used in CANbedded environment.

Q: Is there something to keep in mind combining MICROSAR MEM and OTA software update?

A: Indeed – see hardware specific page.

Q: Is it possible to mix ASR3-based programs/MCAL with MICROSAR MEM?

A: This needs to be checked by our experts.

Q: Is it possible to use Ea with the internal Eep and external Eep?

A: Yes, the Vectors Ea supports more than one Eep driver.

Q: Can I use an external 3rd party Fls/Eep driver together with the Vector Fee/Ea?

A: Yes, if the 3rd party driver follows the ASR4.x specification.

Q: I need more than one Fls or Eep memory driver.

A: This is no typical use case – need to be checked by our experts.

Q: Does an external Fls for OTA software update require a Fee?

A: No, in case of OTA software update, a Fee for the external Fls is not needed.

Q: Memory driver support for a NAND flash memory.

A: Normally, NAND flash memory needs a bad block management in software (this is the most common use case). As we do not have a product for software-based bad block management, we cannot offer a solution for these NAND devices. However, there are some NAND flashes which have a hardware based bad block management, e.g. WinBond W25M/N01 where we provide a flash driver.

© 2022 Vector Informatik GmbH

2 Hardware dependent use cases

2.1 Structure

The use cases are separated into a general information section and seven different use cases which can be combined with each other.

The general information contains all relevant information about the derivative. The use cases are structured as follows:

1. Use Case I: NV Storage Internal Memory

   This use case describes the required components in case that you only want to store Non-Volatile data through the NvM on internal flash. Your project does not contain an HSM or OTA use-case.

> **Use Case II: NV Storage External Memory**

3 NXP S32K3xx

3.1 General Information

- The NXP S32K3xx family uses a different flash technology compared to the S32K1xx family and does not provide a hardware flash EEPROM emulation anymore
- Code and data flash come with separate flash controllers
- All derivatives come with at least two read while write (RWW) partitions for OTA download
- All devices provide hardware A/B swap capabilities
  - A/B swap cuts the flash memory in half
  - HSE (hardware security engine) firmware area is also swapped
- HSE and host core share the same data flash controller
- No vHSM available
- NXP Fls MCAL driver supports both code and data flash as well as external flash memory

3.2 Use Case I: NV Storage Internal Memory

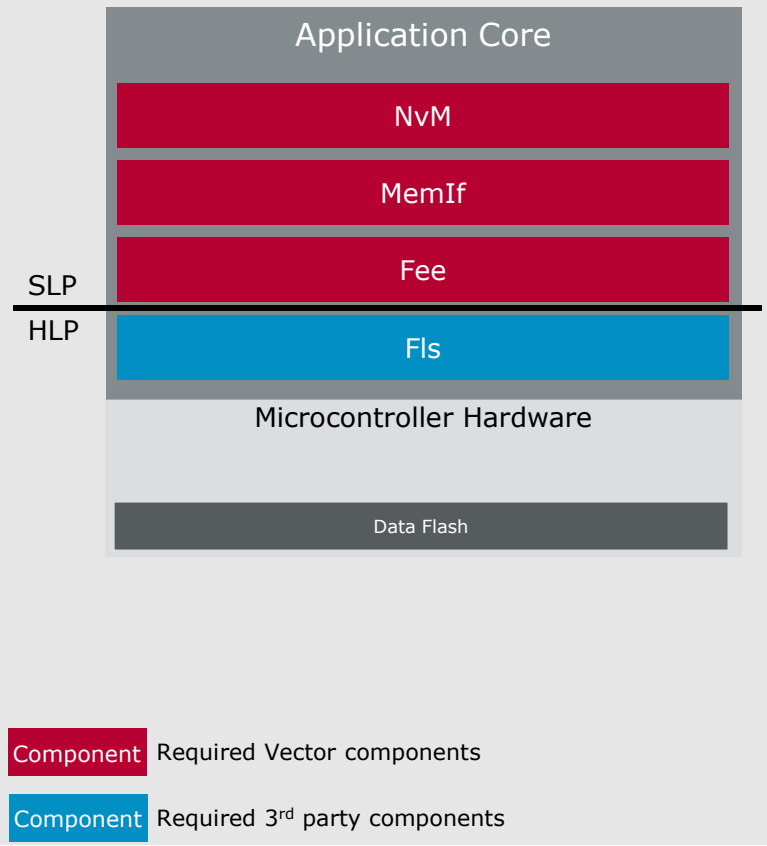


Figure 3-1 Use Case I: NV Storage Internal Memory

Hardware restrictions

> NXP Fls MCAL driver supports both code and data flash as well as external flash memory

> Code and data flash come with separate flash controllers

3.3 Use Case II: NV Storage External Memory

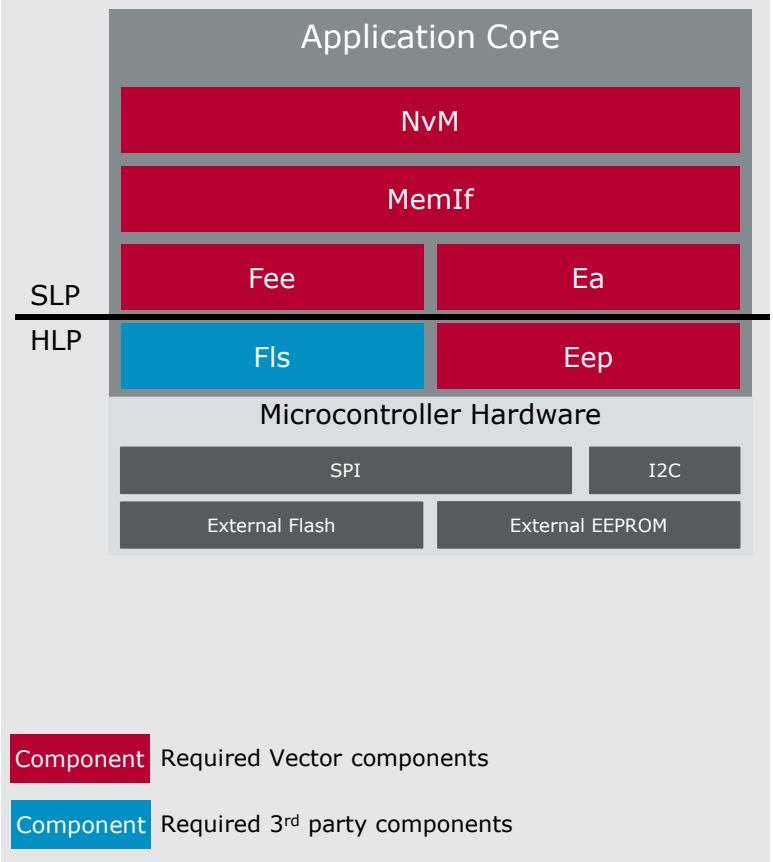


Figure 3-2 Use Case II: NV Storage External Memory

Note
If the external Memory is an EEPROM please choose the right stack Ea and Eep.
In case it's a flash please choose the Fee stack.

Hardware restrictions

> NXP Fls MCAL driver supports both code and data flash as well as external NOR flash memory

> Code and data flash come with separate flash controllers

3.4 Use Case III: OTA Internal Memory and NV Storage Internal Memory

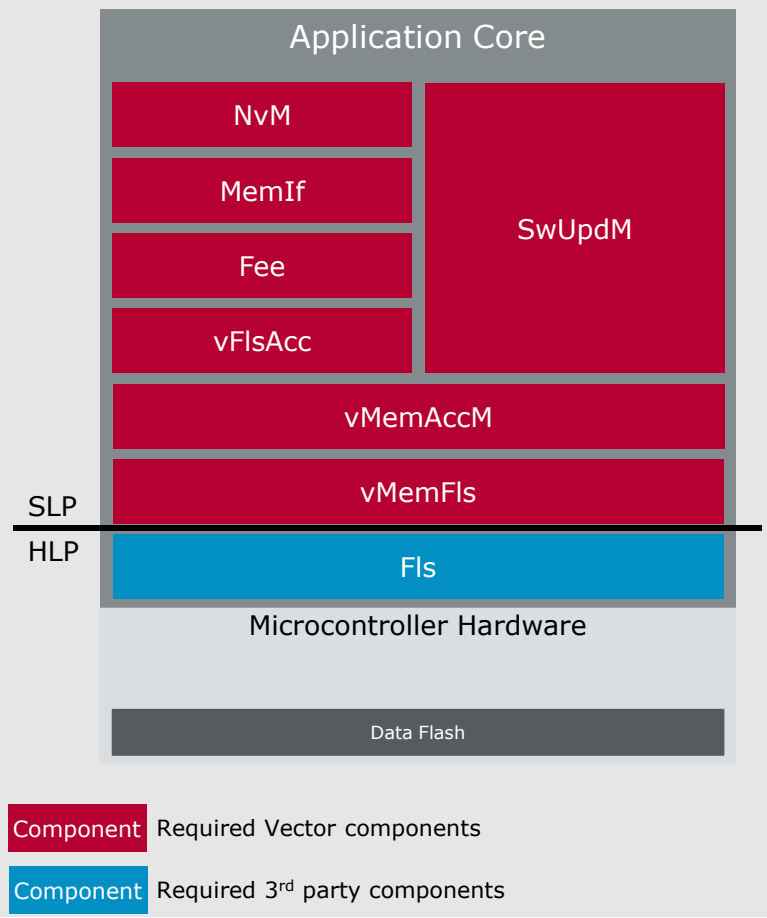


Figure 3-3 Use Case III: OTA Internal Memory and NV Storage Internal Memory

Hardware restrictions

- NXP Fls MCAL driver supports both code and data flash as well as external NOR flash memory
- Code and data flash come with separate flash controllers
- OTA
  - All derivatives come with at least two read while write (RWW) partitions
  - All devices provide hardware A/B swap capabilities
  - A/B swap cuts the flash memory in half

3.5 Use Case IV: OTA External Memory and NV Storage External Memory

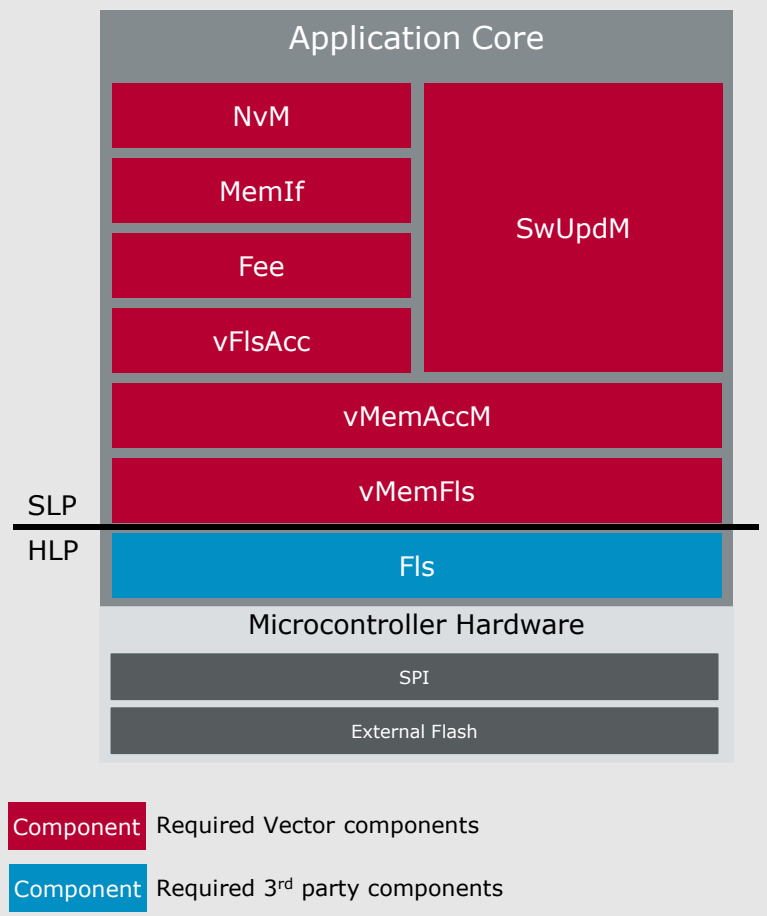


Figure 3-4 Use Case IV: OTA External Memory and NV Storage External Memory

Hardware restrictions

- NXP Fls MCAL driver supports both code and data flash as well as external NOR flash memory
- Code and data flash come with separate flash controllers
- OTA
  - Extended Use-Case
  - All derivatives come with at least two read while write (RWW) partitions
  - All devices provide hardware A/B swap capabilities
  - A/B swap cuts the flash memory in half

3.6 Use Case V: OTA Internal Memory and NV Storage External Memory

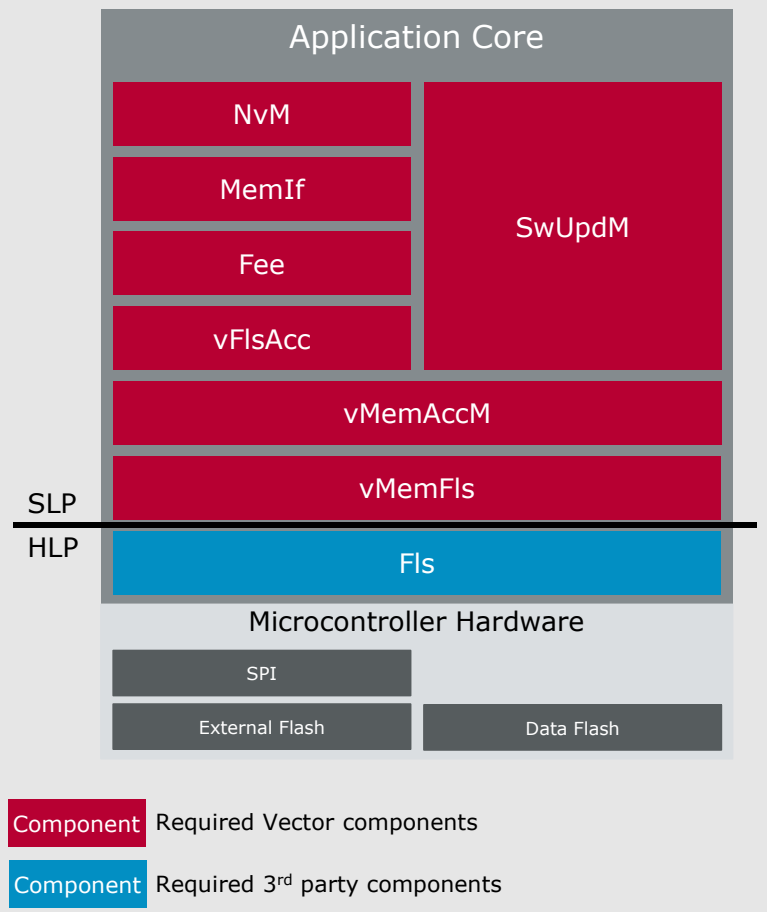


Figure 3-5 Use Case V: OTA Internal Memory and NV Storage External Memory

Hardware restrictions

- NXP Fls MCAL driver supports both code and data flash as well as external NOR flash memory
- Code and data flash come with separate flash controllers
- OTA
  - All derivatives come with at least two read while write (RWW) partitions
  - All devices provide hardware A/B swap capabilities
  - A/B swap cuts the flash memory in half

3.7 Use Case VI: OTA External Memory and NV Storage Internal Memory

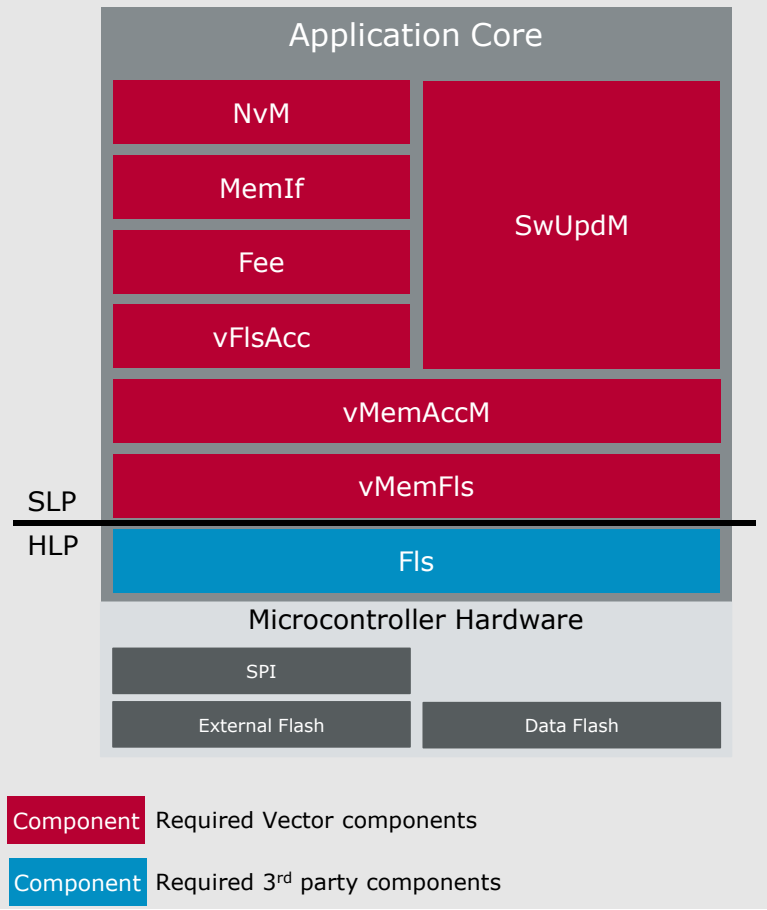


Figure 3-6 Use Case VI: OTA External Memory and NV Storage Internal Memory

Hardware restrictions

- NXP Fls MCAL driver supports both code and data flash as well as external NOR flash memory
- Code and data flash come with separate flash controllers
- OTA
  - All derivatives come with at least two read while write (RWW) partitions
  - All devices provide hardware A/B swap capabilities
  - A/B swap cuts the flash memory in half

3.8 Use Case VII: MSR with HSM

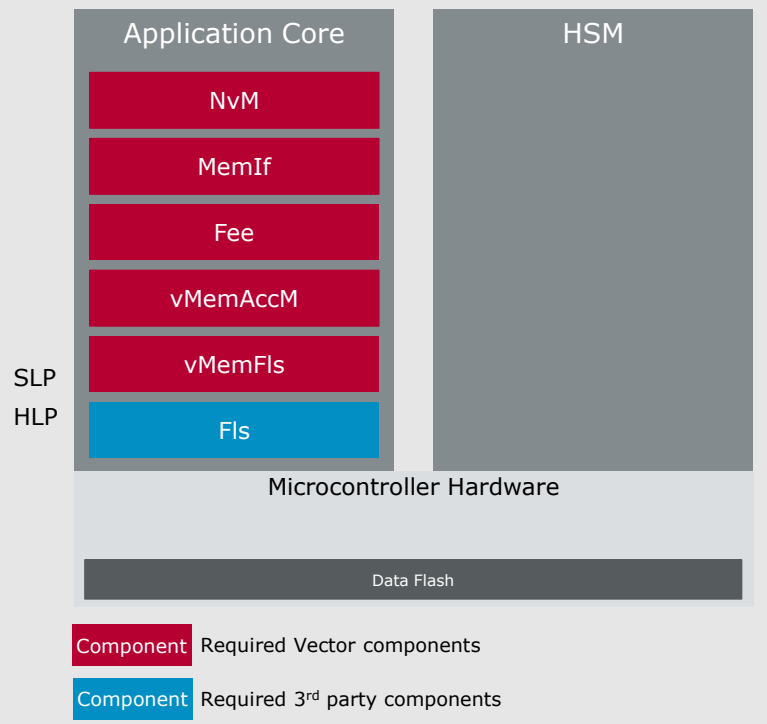


Figure 3-7 Use Case VII: MSR with HSM

Hardware restrictions

> vHSM not available

4 Flash Memory Background Information

4.1 Flash Memory Characteristics

Unlike RAM, flash memory needs to be erased before it can be written

Flash memory is divided in banks (aka flash partition/array), pages and sectors

- Bank is an independent instance of a flash memory
  - Typically, more than one flash bank is embedded in an MCU
- Sector is the smallest erasable portion
  - Typical values > 8-512kBytes
- Page is the smallest writeable portion
  - Typical value 32Bytes – 512Bytes

During a flash write or erase access in a part of a flash bank, it is not possible to read some other part of the flash bank

Flash is managed by a flash controller that implements a state machine to control the erase and programming operation

- Separate flash controllers allow independent accessibility of flash banks (aka read while write – RWW) for OTA use case



© 2022 Vector Informatik GmbH

Figure 4-1 Flash Memory Characteristics I/II

Flash wear out

- Erasing and writing put stress to the flash memory cells
  - High voltage is applied to the flash cells
  - Limited erase/write cycles
- Flash memory utilization needs to be balanced
  - Wear levelling/round robin algorithm needs to be in place
  - Is implemented by Fee for data storage

Flash memory comes in two implementations

- NOR
  - Standard memory for classic embedded ECUs
  - Either MCU-internal or as external memory device
- NAND
  - Typically used with SoCs in combination with POSIX OS
  - Only as external memory device

4.2 Flash Memory Types – NOR

Typical embedded NV memory type of current microcontrollers

- Provides random memory access
- Allows direct code execution and data access
- Low latency due to direct memory addressing
- High reliability

Typical characteristics

- Write performance < 500kByte/s
- Memory size up to 16MByte for internal flash and up to 256MByte for external flash
- Erase/write cycles ~ 10k

Higher cost compared to NAND flash memory due to larger flash cell size

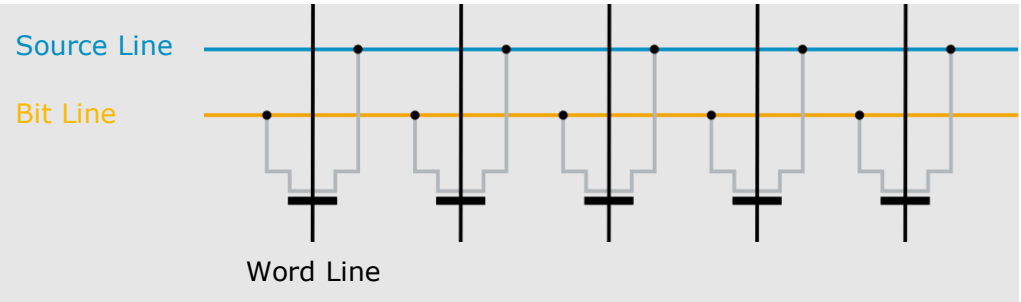


Figure 4-2 Flash Memory Types – NOR

4.3 Flash Memory Types – NAND

Implemented as external devices and used with microprocessor/SoCs, typically in combination with a POSIX OS and a file system

- Provides **block based** memory access
- No direct code execution and data access
- Higher latency for random access
- Less reliable, bad block management must be implemented

Typical characteristics

- Write performance in MByte/s range
- Memory size in GByte range
- Erase/write cycles ~ 100k

Very cost sensitive due to small cell size, especially with multi level flash cells

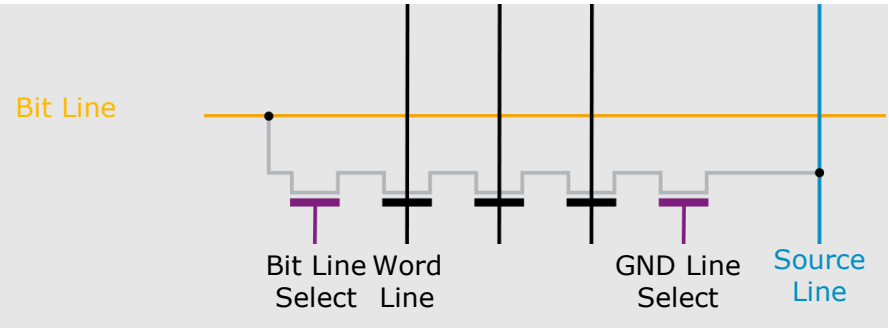


Figure 4-3 Flash Memory Types – NAND

© 2022 Vector Informatik GmbH

4.4 Memory Types – NAND Flash Implementations

NAND flash comes in two flavors

- Unmanaged
- Managed like (e)MMC, SD Card, UFS or SSD

Unmanaged NAND flash

- Very cheap
- Bad block management and wear leveling needs to be implemented in software
- SoC initial boot code needs to be aware of bad block handling

Managed NAND flash

- More expensive compared to unmanaged NAND flash
- Bad block management, wear levelling and command queuing implemented in hardware flash controller
- Very high data transfer rates depending on flash controller and memory topology

© 2022 Vector Informatik GmbH

5 Glossary and Abbreviations

5.1 Abbreviations

Abbreviation	Description
ASR	AUTOSAR
BSW	Basis Software
D Flash	Data Flash
Ea	EEPROM Abstraction
Eep	EEPROM Driver
FBL	Flash Boot Loader
Fee	Flash EEPROM Emulation
Fls	Flash driver
HSE	Hardware Security Engine
HSM	Hardware Security Module
IPC	Inter Process Communication
MCAL	Microcontroller Abstraction Layer
MemIf	Memory Interface
MICROSAR AP	MICROSAR Adaptive
MSR	MICROSAR
NV	Non-Volatile
NvM	Non-Volatile Ram Manager
OTA Update	Over The Air Update
P Flash	Program Flash
RAM	Random Access Memory
ROM	Read Only Memory
SecOc	Secure Onboard Communication
SoC	System on Chip
SPI	Serial Peripheral Interface
PCM	Phase Change Memory
RWW	Read While Write

6 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2022 Vector Informatik GmbH